CS637: Parallel Computing Lab Assignment

Parallel BOGO Sort using OpenMP

Department of Computer Engineering

OpenMP Concepts: parallel, for, sections, critical, atomic, barrier, thread control

---

Background
BOGO Sort is a highly inefficient sorting algorithm based on randomness. It repeatedly shuffles the array until it becomes sorted. Although impractical for real-world sorting, BOGO Sort is useful for understanding **parallel trial-based computation**, where multiple threads attempt the same task simultaneously and the first successful result terminates the process.

BOGO Sort Algorithm (Definition)
Given an array A of size N:

1. Check if A is sorted.
2. If sorted, stop.
3. Otherwise, shuffle A randomly and repeat.

Problem: Parallel BOGO Sort Performance Study

Objective
Implement BOGO Sort in both sequential and parallel form. Run the parallel version using **2 threads, 4 threads, and 8 threads**, and compare execution times, number of shuffles, and speedup.

Input
A randomly generated integer array of size:

$$N = 100$$

Values should be unique and range from 1 to 50.

Output
Your program must print:

- the original unsorted array
- the sorted array
- number of shuffles attempted
- time taken (in seconds)

Tasks and Requirements

1. **Parallel Implementation (OpenMP)**
   Implement a parallel BOGO Sort using OpenMP such that:
   - each thread works independently by shuffling its own copy of the array
   - the first thread that finds a sorted array signals others to stop
2. **Thread Experiments**
   Run the parallel version with:
   $$T = 2,\ 4,\ 8$$
   and record results.
3. **Termination Condition**
   Use a shared flag variable such as:
   $$found = 0$$
   When a thread finds a sorted array, it must:
   - copy the sorted array into a shared result array
   - set `found = 1`
   - stop all other threads from continuing
4. **Synchronization Requirement**
   Since multiple threads may attempt to update the shared result, you must use:
   - `#pragma omp critical` or `#pragma omp atomic`
5. **Statistics Table**
   Your final output must contain a summary table showing:

Threads	Time (sec)	Shuffles	Speedup
2			
4			
8			

Expected Output Format (Example)

```
Sample Array: 5 3 8 1 2 7 6 4

Parallel Run (Threads = 4):
Thread 2 found the sorted array!
Sorted Array: 1 2 3 4 5 6 7 8
Time Taken: 1.140 sec
Total Shuffles: 108210

Summary Table:
Threads    Time(sec)    Shuffles    Speedup
2          1.980        104122      1.77
4          1.140        108210      3.08
8          0.820        110050      4.28
```

End of Assignment